```java
//Tasneem Ibrahim Hassan Mohamed
//22101382

import java.util.Scanner;

    class GraphTheory {

    //Attributes
    private int vertexNumber;
    private boolean[][] adjacencyMatrix;

    //constructor
    public GraphTheory(int vertexNumber) {
        this.vertexNumber = vertexNumber;
        this.adjacencyMatrix = new boolean[vertexNumber][vertexNumber];
    }

    //methode for Adding edges
    public void AddEdge(int source, int destination) {
        adjacencyMatrix[source][destination] = true;
    }

    //checking if it is Reflexive
    public boolean Reflexive(int[][] matrix) {
        for (int i = 0; i < matrix.length; i++) {
            if (matrix[i][i] != 1) {
                return false;
            }
        }
        return true;
    }

    //checking if it is Symmetric
    public boolean Symmetric(int[][] matrix) {
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix.length; j++) {
                if (matrix[i][j] != matrix[j][i]) {
                    return false;
                }
```

```
40              }
41          }
42          return true;
43      }
44
45      //checking if is Transitive
46      public boolean Transitive(int[][] matrix) {
47          int n = matrix.length;
48          int[][] result = new int[n][n];
49          for (int i = 0; i < n; i++) {
50              for (int j = 0; j < n; j++) {
51                  for (int k = 0; k < n; k++) {
52                      result[i][j] += matrix[i][k] *
    matrix[k][j];
53                  }
54              }
55          }
56          for (int i = 0; i < n; i++) {
57              for (int j = 0; j < n; j++) {
58                  if (result[i][j] > 0 && matrix[i][j
    ] == 0)
59                      return false;
60              }
61          }
62          return true;
63      }
64      //checking if it is Acyclic
65      public boolean Acyclicity(int[][] matrix) {
66          boolean [] visited = new boolean [matrix.
    length];
67          boolean[] recursion = new boolean[matrix.
    length];
68          for (int i = 0; i <matrix.length; i++) {
69              if (hasCycle(matrix,1, visited, recursion
    )) {
70                  return false;
71              }
72          }
73          return true;
74      }
75
```

```java
    //checking if it has Cycle
    private boolean hasCycle(int[][] matrix, int vertex, boolean[] visited,  boolean[] recursionStack) {
        if (recursionStack[vertex])
            return true;
        if (visited[vertex])
            return false;
        visited[vertex] = true;
        recursionStack[vertex] = true;
        for (int i = 0; i < matrix.length; i++) {
            if (matrix[vertex][i] == 1 && hasCycle(matrix, i, visited, recursionStack))
                return true;
        }
        recursionStack[vertex] = false;
        return false;
    }
    //checking if it is connected
    public boolean Connectivity(int[][] matrix) {
        boolean[] visited= new boolean[matrix.length];
        DFS(matrix, 0, visited);

        for (int i = 0; i < matrix.length; i++) {
            if (!visited[i]) {
                return false;
            }
        }
        return true;
    }

    private static void DFS(int[][] matrix, int vertex, boolean[] visited) {
        visited[vertex] = true;
        for (int i = 0; i < matrix.length; i++) {
            if (matrix[vertex][i] ==1 && !visited[i]) {
                DFS(matrix, i, visited);
            }
        }
```

```
111     }
112     //checking if it has Eulerian path
113     public boolean EulerianPath(int[][] matrix) {
114         int n = matrix.length;
115         int[] degree = new int[n];
116         int odd = 0;
117         for (int i = 0; i < n; i++) {
118             for (int j = 0; j < n; j++) {
119                 degree[i] += matrix[i][j];
120             }
121             if (degree[i] % 2 != 0)
122                 odd++;
123         }         if (odd == 0 || odd == 2)
124             return true;
125         else
126             return false;
127     }
128
129     public static void main(String[] args) {
130         Scanner Tasneem = new Scanner(System.in);
131
132         System.out.print("Enter number of vertices
    : ");
133         int vertexNumber = Tasneem.nextInt();
134         int[][] matrix = new int[vertexNumber][
    vertexNumber];
135
136         GraphTheory G1 = new GraphTheory(
    vertexNumber); //creating object
137
138         System.out.println("Enter your adjacency or
    Incident matrix : ");
139         for (int i = 0; i < vertexNumber; i++) {
140             for (int j = 0; j < vertexNumber; j++) {
141                 matrix[i][j] = Tasneem.nextInt();
142
143
144             }
145         }
146         //printing data of the graph
147         System.out.println("Properties of the Graph
```

```
147  ---> ");
148          System.out.println("Reflexivity is " + G1.
     Reflexive(matrix));
149          System.out.println("Symmetric is " + G1.
     Symmetric(matrix));
150          System.out.println("Transitivity is " + G1.
     Transitive(matrix));
151          System.out.println("Acyclicity is " + G1.
     Acyclicity(matrix));
152          System.out.println("Connectivity is " + G1.
     Connectivity(matrix));
153          System.out.println("Has Eulerian Path : " +
     G1.EulerianPath(matrix));
154      }
155
156 }
```